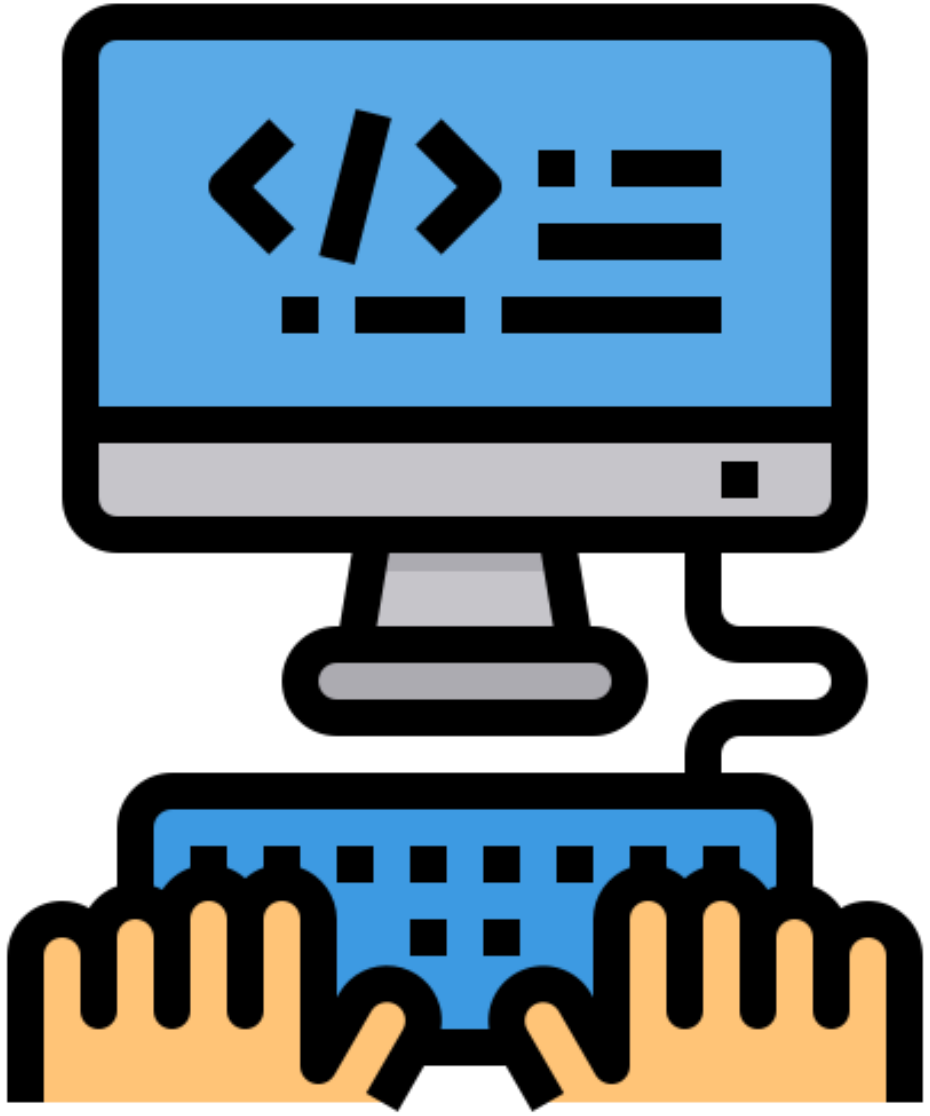
CS 24 AP Computer Science A Review

Week 10: FRQ Problems

DR. ERIC CHOU
IEEE SENIOR MEMBER

Topics

- Print List
- Print Justified Lines
- Find Pattern locations



Print List

SECTION 1

Problem

Given a list `a = [1, 2, 3]`, how to print this list a as

```
[1, 2, 3]
```

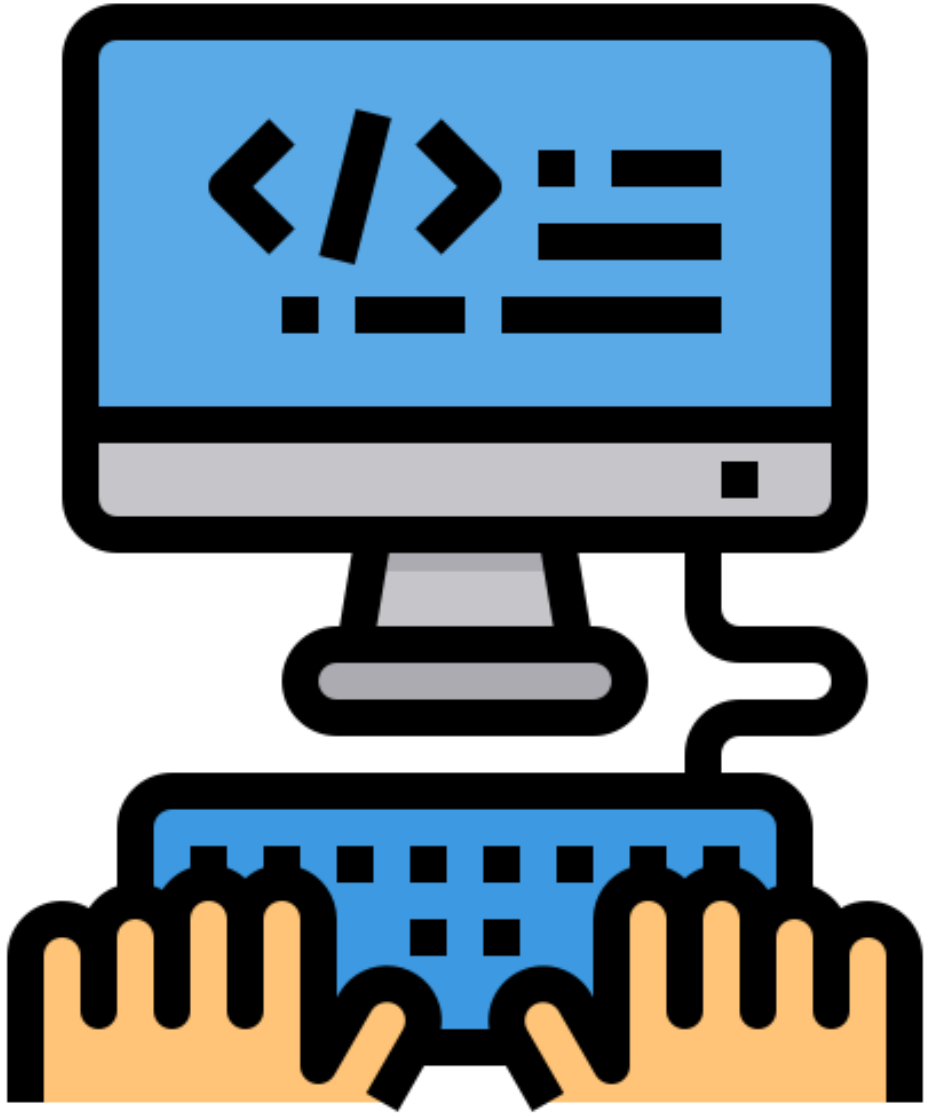
?

Algorithm I

```
public String printList(int[] a){
    String s = "[";
    for (int i=0; i<a.length; i++){
      if (i==0) s += a[i];
      else s += ", "+a[i];
    }
    s += "]";
    return s;
}
```

Algorithm II

```
public String printList(int[] a){
    String s = "[";
    for (int i=0; i<a.length; i++){
      if (i==a.length-1) s += a[i];
      else s += a[i]+", ";
    }
    s += "]";
    return s;
}
```



Print Justified Lines

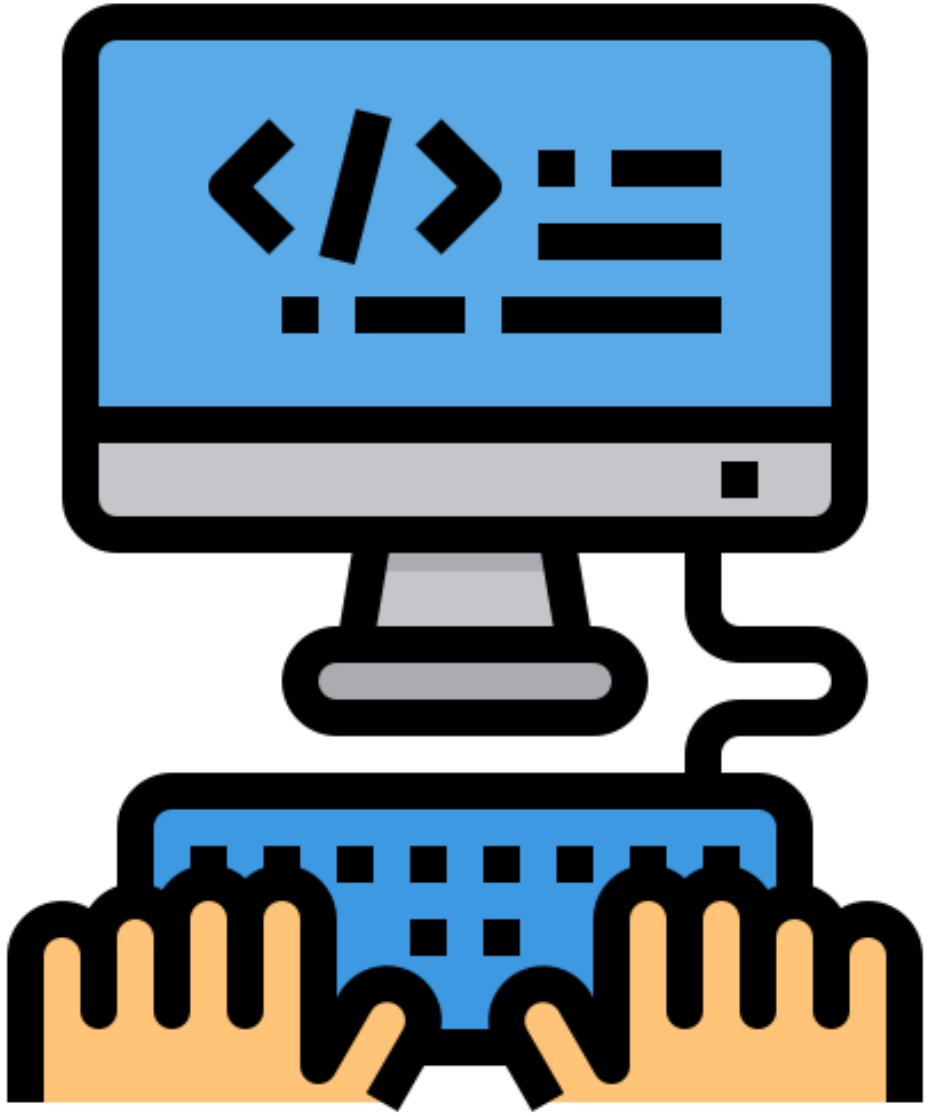
SECTION 2

Print Justified Line

If a line is of fixed size which is longer than all the words combined length + words.length-1; How to print a line in justified format?

Algorithm I

```
public String printLine(String[] words, int line){
  int sum = 0;
  for (String s: words) sum += s.length();
  line -= sum;
  line -= words.length-1;
  int gaps = line/(words.length-1); line -= gaps * (words.length-1);
  String r= “”;
  for (int i=0; i<words.length; i++){
    if (i==words.length-1) r += words[i];
    else { r += words[i] + “ “;
           for (int j=0; j< gaps; j++){
               r += “ “;
           }
           if (line >0){
             r += “ “;
             line -= 1;
           }
    }
  }
  return r;
}
```



Pattern List

SECTION 3

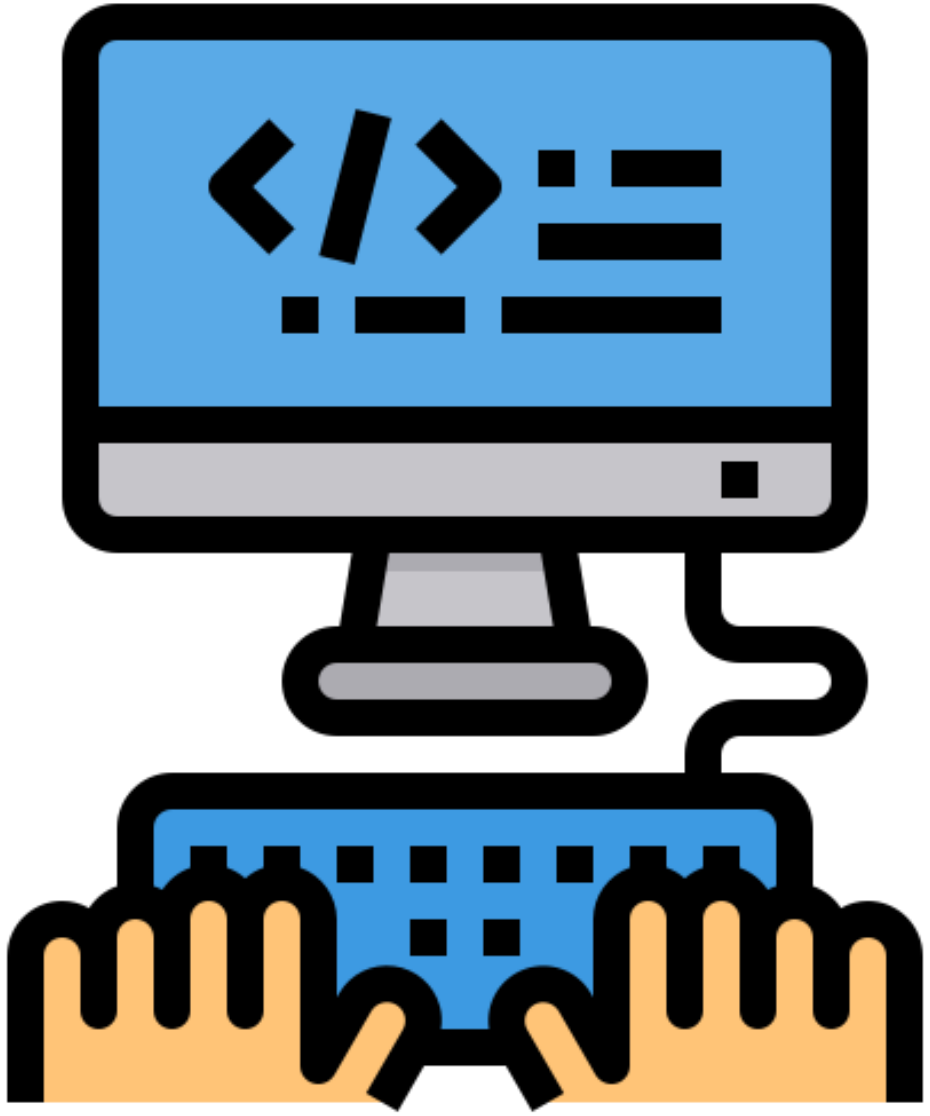
Problem

Find a string pattern pat in a string s (non-overlapping)?

Algorithm I

```
public ArrayList<Integer> printList(String s,
                                    String pat){
    ArrayList<Integer> idx = new ArrayList<Integer>();

    int p = s.indexOf(pat);
    while (p >=0){
        idx.add(p);
        p = s.indexOf(s, p+pat.length());
     }
     return idx;
}
```



Other Algorithms

SECTION 4

Problem

Looping Through an ArrayList and Seeing if Your Object Matches

Example: We have a method to check if how many Strings in an ArrayList, StringList, is equal to a String passed into the parameters.

Algorithm

```
public int countSame(String check) {
    int count = 0;
    for(int i=0; i<StringList.size(); i++) {
    if(StringList.get(i).equals(check) {
        count++;
        }
    }
    return(count);
}
```

Problem

Looping Through an ArrayList and Compare Adjacent Values

Example: We have a method that checks and returns true if any adjacent Strings in an ArrayList, StringList, are equal (two of the same Strings one after another).

Algorithm

```
public boolean adjacentStrings() {
    for(int i=0; i<StringList.size(); i++) {
        if(StringList.get(i).equals(StringList.get(i-1)) {
            return(true);
        }
    }
    return(false);
}
```

Discussion

This algorithm goes through every list item except the first one, and checks if it is equal to the list item before it. If the program finds one that is equal, it returns true. If it goes through every item and doesn’t find any that are equal, it returns false. The loop starts at i=1 because it compares the second item (i=1) with the first one(i=0).

Problem

Looking Through a List and Removing an Object that Meets a Criteria

Example: We have an ArrayList of Dog objects. In the Dog class, you see that each Dog has a getBreed() method that returns a String of the Dog's breed. This method will remove each Dog in an ArrayList of Dogs, dogList, that are the same breed as the one specified in the parameter.

Algorithm

```
public void removeDogBreed(String breedToRemove) {
    for(int i=0;i<dogList.size();i++) {
        if(dogList.get(i).getBreed().equals(breedToRemove) {
            dogList.remove(i)
            i-=1;
        }
    }
}
```

Discussion

i-=1 is necessary because when you remove a Dog, the list shifts, so you need to subtract one from i to make sure it doesn’t skip checking any Dogs.

Problem

Finding the Smallest or Largest Number Value in an ArrayList

Example: Let’s say you have an ArrayList of Dogs, dogList. The class documentation shows that the Dog class has a method getAge() which returns the age of the Dog. We have a method that returns the Dog that is the youngest. If there are two Dogs of the same age, return any.

Algorithm

```
public Dog getYoungestDog() {
    int youngestAge = dogList.get(0).getAge();
    Dog youngestDog = dogList.get(0);
    for(int i=1; i<dogList.size();i++) {
        if(dogList.get(i).getAge() < youngestAge) {
            youngestAge = dogList.get(i).getAge();
            youngestDog = dogList.get(i);
        }
    }
    return(youngestDog);
}
```

Problem

Question Where You Need to Manipulate Individual Digits in a Number

Example: a method that checks if the number passed through a parameter is one of the digits of an int num. Assuming that the number passed in cannot be 0.

→ If num is 1938 and the number passed in the parameter is 3, return true.

→ If num is 1938 and the number passed in the parameter is 4, return false.

Algorithm

```
public boolean isIn(int checkDigit) {
    while(num>0) {
        if(num%10==checkDigit) {
            return(true);
        }
        num/=10;
    }
    return(false);
}
```

Discussion

This algorithm is useful if they want you to traverse a number without converting it to a String. To understand the logic behind it, pretend our number is 1938. num%10 is equal to 8, since % gives the remainder, and when 1938 is divided by 10, the remainder is 8. If num%10 is equal to checkDigit, return true. If it's not, divide 1938 by 10 and set that equal to num. Since both are int types, the program will return an answer without the remainder. 1938/10 = 193. this is your new num. Repeat the loop. If 193%10 is equal to checkDigit, return true. Repeat the process, until you divide your last digit and end up with 0. If you are here, nothing returned true, so checkDigit is not in num. Now return false.

Discussion

The College Board loves asking questions about digits in a number. Although they may not ask it in the same context as this question, understanding how to use % and / by 10 to check digits will be useful.

Problem

Checking if Numbers (Or Other Comparable Objects Such as Strings) are in Ascending Order

For putting Strings into ascending order, you can use String.compareTo() method instead of the < or > operators used for numbers.

Example: We have a method that returns false if an ArrayList, numList, is in strictly ascending order, and false otherwise.

Algorithm

```
public boolean isAscending() {
    for(int i=1; i<numList.size(); i++) {
        if(numList.get(i)<numList.get(i-1)) {
            return(false);
        }
    }
    return(true);
}
```

Discussion

In this example, we first loop through each item. If any of the items are smaller than the one before it, the ArrayList is not in ascending order, so return false. If you get through all the numbers without returning false, return true because it means everything is in ascending order.

Problem

Inserting a Number (Or Other Comparable Objects Such as Strings) in its Appropriate Place

Example: We have a method that needs to insert a number into an ArrayList, numList, into its appropriate place. The ArrayList is sorted by ascending order, so we want to put the number, num, into the ArrayList to keep the ascending order.

Algorithm

```
public void insertInPlace(int num) {
    for(int i=0; i<numList.size();i++) {
        if(num<numList.get(i)) {
            numList.add(i,num);
            return;
        }
    }
    numList.add(num);
}
```

Discussion

In this example, we go through the list until you find a number that your num is smaller than, meaning your num belongs before it — the last numList.add() is to add the num in case your num is bigger than everything in the list.

Problem

Removing a Single Character from a String

Example: We have a method that will remove all instances of the String smallWord in a String word.

Algorithm

```
public void removeLetter(String smallWord) {
    int start = word.indexOf(smallWord);
    int l = smallWord.length();
    while(start>-1) {
        word = word.substring(0,start)+
               word.substring(start+l);
        start = word.indexOf(smallWord);
    }
}
```

Discussion

Here, we find the index of a word, and remove it from a String until it no longer exists. Note: this collapses a word, which may or may not be what you want.

Problem

Reversing a String

Example: We have a method that will take a String and return a reversed version.

Algorithm

```
public String reverse(String reverseMe) {
    String reversed = "";
    for(int i=reverseMe.length()-1; i>=0; i--) {
        reversed = reversed +
                   reverseMe.substring(i, i+1);
    }
    return(reversed);
}
```

Discussion

Here, we loop through the string but from the end to the front, and add letters as we go.

Problem

Checking if a String Contains Numbers or Vowels (Or Anything You Want)

Example: We have a method that returns true if our String stringCheck has numbers in it and false if not. For example, “wilson” would return false, and “wilson10” would return true.

Algorithm

```
public boolean hasNum(String stringCheck) {
    String numbers = “0123456789”;
    for(int i=0; i<stringCheck.length(); i++) {
        String letter = stringCheck.substring(i,i+1);
        if(numbers.indexOf(letter)>-1) {
            return(true);
        }
    }
    return(false);
}
```

Discussion

Here, we loop through every character in stringCheck. Then, we check if the letter is contained in the String “0123456789”. If it is, then that “letter” is a number. Return true. If we finish checking and that doesn’t happen, return false.

Problem

Detecting duplicates means determining if an array contains at least one value more than once.

Algorithm

```
public static boolean hasDuplicate(int[] vals)
{
    for(int i = 0; i < vals.length - 1; i++)
    {
        for(int j = i + 1; j < vals.length; j++)
        {
            if(vals[i] == vals[j])
                return true;
        }
    }

    return false;
}
```

Discussion

Skipping the last element in the outer loop is not strictly required. If the outer loop is allowed to run up to and including the last element, the inner loop will not run when the outer loop visits the last element.

The inner loop cannot skip the last element.